数据库设计(Database/DB)

1. ACID 原则：Atomicity 原子性，Consistency 一致性， Isolation 隔离性，Durability 持久性

"A""atomicity"原子性–全有全无　　"C""consistency"一致性–所有规则都保持不变

"I" "isolation"隔离性–所有事务独立执行　"D" "durability"持久性–对电源短缺、故障具有鲁棒性

2.数据库术语

1)实体(Entity): 实体是可以存储数据的单个对象.它是表的"主题".

2)属性(Attribute):与数据库对象有关的单个数据项.数据库模式将一 or 多个属性与每个数据库实体相关联.属性也称为字段或列.

3)集(Set):一种对象的集合,该对象被称为集合的元素,以这样一种方式指定:我们可以原则上判断给定的对象是否属于它.集合中元素的顺序和重复无关紧要.**无序可重复**。因此，例如，{1，2，3}={3，2，1}={1，3，1，2，2}。

4)列表(List): 一种数据结构,它包含许多可能是不同类型的值,通常从头到尾进行顺序访问,即"**有序**列表".

5)元数据(Metadata):是"关于数据的数据".该术语是指有关数据本身的信息.

6)模式(Schema): 描述数据库中关系的元数据集合.它可以简单地描述为数据库的"布局"或概述将数据组织到表中的方式的蓝图.

7)元组(Tuple): 一条记录,表或关系中的一**行**(实现时)."元组"和"实体"不同,元组=记录或表行,实体=概念性事物或事件

8) Relation: 表 Relationship:表间关系或实体间关系 关系模式静态，关系动态

3.ER 图

ER 图是一个表示实体\属性和它们之间关系的图形化工具.它主要由三个元素组成:实体\属性和关系.关系是指实体之间的联系或联系类型,如一对多\多对多等.（实体集：用矩形表示弱实体集及其关系用**双边界**；属性：用椭圆表示；关系：用菱形表示；一对一（双箭头）、多对一（一那端有箭头）、多对多（无箭头）；子类：由三角形表示，连接到顶点的是父实体，底部是子实体（继承其父属性的任何键，它也可能有自己的键））

1)将 E / R 图设计转换为关系数据库:多对多:E 主\F 主\R 的属性/多对一:E 所有\F 主\R 的属性/一对一;所有属性,主键选一个/子类:根主+自身属性/弱实体集:同多对一

2)常用约束

键约束:属性或属性集在关系表中必须具有唯一且不为空的值.

单值约束:实体集的每个属性都有一个值.**键是单值约束的主要来源.**非键单值属性可以采用 null 值(键不能为 null) **多对一关系，多方单值约束，一方主键约束 主键约束一定是单值约束**

域约束: 每个属性 X 的值必须是 X 域中的原子值.一般限制条件:在数据库中应保留的其他任意约束,如检查约束和范围约束.引用完整性约束:要求某个对象引用的值实际存在于数据库中.

每个实体集都必须有一个键,弱实体集通常意味着多对一关系(包括作为特例的一对一关系)和引用完整性.

4.一二三范式、BNCF

第一范式（1NF）：每个元组的每个组成部分都是原子性的值。

第二范式（2NF）：允许关系表中的传递 FD，但禁止左侧为键的适当子集的非平凡 FD

第三范式（3NF）：关系表 R 符合第三范式（3NF）：当 A1, A2, ⋯, An→B 是非平凡 FD 时，{A1, A2, ⋯, An}是一个超码，或者 B 是某个键的成员。

Boyce-Codd 范式（BCNF）：关系表 R 是符合 BCNF 范式，当且仅当每当 R 有一个非平凡的 FD：A1, A2, ⋯, An →B ，那么{A1, A2, ⋯, An }是 R 的一个超码。 也就是说，**每个非平凡 FD 的左边必须是一个超键。**

1NF：表中的每个字段都是最小的数据单元

例如，用户信息中，"联系方式"字段不应该为"手机+居住地址"，而是单独作为两个原子值字段。否则当需要针对"手机"进行查询时，不管是查询性能和成本都是不理想的。

2NF：在 1NF 的基础上，表中所有的非码属性必须完全依赖于候选码

因为我们知道在一个订单中可以订购多种产品，所以单单一个 OrderID 是不足以成为主键的，主键应该是（OrderID，ProductID）。显而易见 Discount（折扣），Quantity（数量）完全依赖（取决）于主键（OderID，ProductID），而 UnitPrice，ProductName 只依赖于 ProductID。所以 OrderDetail 表不符合 2NF。不符合 2NF 的设计容易产生冗余数据。

3NF：在 2NF 的基础上，非主属性之间没有相互依赖（消除传递依赖）

考虑一个订单表【Order】(OrderID, OrderDate, CustomerID, CustomerName, CustomerAddr, CustomerCity)主键是（OrderID）。 其中 OrderDate，CustomerID，CustomerName，CustomerAddr，CustomerCity 等非主键列都完全依赖于主键（OrderID），所以符合 2NF。不过问题是 CustomerName，CustomerAddr，CustomerCity 直接依赖的是 CustomerID（非主键列），而不是直接依赖于主键，它是通过传递才依赖于主键，所以不符合 3NF。

3NF 不允许非主属性被另一个非主属性决定，但允许主属性被主属性决定，而 BCNF 中，任何属性（包括非主属性和主属性）都不能被主属性所决定（主属性内部也不能部分或传递依赖）。

函数依赖(FD)：“如果 R 的两个元组在属性 A1, A2, ⋯, An 上一致（即，这些元组在各自的组件中对于这些属性中的每一个都具有相同的值），那么他们一定也在另一个属性 B 上一致。”写为 A1, A2, ⋯, An→B

候选码(Candidate key): 最小超码，可以唯一地标识单个记录的一组属性，但不一定指定为键

主属性（Prime Attribute）：作为候选键的成员的属性　**非主属性：**不包含在任何一个候选码中的属性称为非主属性

超码（Super Key): 由一个属性或者多个属性组成的集合，这些属性的组合能唯一标识一个实体

平凡函数依赖: 如果 B 是 A 中的一个，An→B 是平凡的（例如，学生 ID 和学生姓名->学生姓名）

5.**数据结构**:数据的逻辑结构:反映数据元素之间的逻辑关系;数据的物理结构:数据结构在计算机中的表示;数据存储结构:常用的存储结构有顺序存储\链式存储\索引存储和哈希存储等.

数据类型:精确数字:TINYINT\SMALLINT\INT\BIGINT\DECIMAL(p,s)

近似数字:FLOAT\REAL; 字符串:CHAR(n)– 固定长度\VARCHAR(n)– 可变长度\TEXT;Unicode 字符串:NCHAR(n)\NVARCHAR(n)\NTEXT; 日期和时间:时间:hh:mm:ss\ 日期:YYYY-mm-DD\ 日期时间:YYYY-mm-DD hh:mm:ss\日期时间 2:YYYY-mm-DD hh:mm:ss

6.其他：数据库主要特征：模式（文件系统的设计或结构，描述系统中不同元素之间的关系和数据的层次结构）、查询、大储存空间、控制访问；数据库管理系统(Database Management System/DBMS)；数据仓库会随着旧数据库的更改而更新，但不一定是实时的

SQL 语句

1.**简单 SQL 查询**

分组: GROUP BY 子句对 SELECT 语句的结果分组.可以按照一个或多个列的值对结果进行分组,从而实现基于聚合函数的统计计算.

排序:ORDER BY 子句对 SELECT 语句的结果进行排序.可以按照一个或多个列的值进行升序或降序(DESC)排序.

统计:使用聚合函数(如 **COUNT\SUM\AVG\MIN\MAX**)对 SELECT 语句的结果进行统计计算.这些函数可以对整个结果集或分组后的结果进行计算.COUNT 适用于副本,除非另有说明;

使用聚合时,SELECT 只能有两种类型的表达式:GROUP BY 子句中的属性+聚合

运算符:SQL 支持各种运算符,包括算术运算符\比较运算符\逻辑运算符等.这些运算符可以用于 WHERE 子句中的条件表达式,从而实现数据的筛选和过滤.

字符串处理:SQL 支持各种字符串函数,包括 CONCAT\SUBSTRING\UPPER\LOWER 等.这些函数可以对字符串进行拼接\截取\转换大小写等操作,从而实现对字符串的处理.

查询:SELECT attributes　FROM tables　WHERE conditions

SELECT:投影数据.这可以是现有字段的列表或某些表达式.

FROM:关系/表格.这可以是现有关系\临时表和子查询.

WHERE:选择数据.限制查询返回的行的条件表达式.

LIKE 运算符:根据模式匹配比较两个字符串.表达式:s LIKE p, s 是字符串,p 是一种模式（_:任意单个字符;%:任意字符序列）

DISTINCT 关键字:删除查询结果中的重复项

内连接意味着我们只返回两个表的"ON"子句都为真的行.也就是说,只返回两个表中都匹配的行.（交叉、θ、自然连接）

外部连接（只能在 from 子句中指定）：

全外连接 Full outer join:包括两个表中的所有行,不管是否找到匹配项

左外连接 Left outer join:包括 JOIN 子句左边指定的表中的所有行

右外连接 Right outer join:包含 JOIN 子句右侧指定的表中的所有行

运算符优先级:计算运算>比较运算>NOT>AND>(BETWEEN,IN,LIKE,OR)>(=Assignment)

HAVING 子句: HAVING <条件>可以跟在 GROUP BY 子句后面。条件适用于每个组，不满足<条件>的组将被排除可以引用属性,只要该属性在组中有意义,GROUP BY 子句中的属性+聚合

数据转换函数:Convert :SELECT CONVERT(DECIMAL(4,2), height)　Cast :SELECT CAST(height AS DECIMAL(4,2))　Datepart:Select DATEPART(MONTH, timestamp)

定义主键： 在后面直接标注 PRIMARY KEY 或者 PRIMARY KEY()

增删列 ALTER TABLE　ADD/DROP　**插入** INSERT INTO （）VALUES（）按顺序给值,没有则为 NULL

UNION 并集 **INTERSECT（交集）**返回由左输入查询和右输入查询同时返回的非重复行.**EXCEPT（差集）**从左输入查询中返回右输入查询没有找到的所有非重复行.

声明外键 Cname CHAR(30) REFERENCES Company(CName) 或 CName CHAR(30), FOREIGN KEY (Cname) REFERENCES Company(CName)

2. **数据库窗口函数**是一种用于对查询结果进行分组\排序\聚合和排名等操作的函数.它可以在结果集中创建一个"窗口",并对窗口中的数据执行聚合函数,如 SUM\AVG\COUNT 等.窗口函数还可以使用 OVER 子句来控制窗口的大小\排序和分组方式.

SELECT ROW_NUMBER() OVER (PARTITION BY <attributes>ORDER BY <attributes>) AS name　FROM relation

窗口函数根据对查询结果的某些操作为每一行返回一个值.允许更复杂的数据分类和排序,包括难以捉摸的分位数函数

ROW_NUMBER():依据组显示每一条记录在该组中出现的位置,比如:若有两个第一名.则这两个第一名在一组内排名为 1\2,下一组的排序仍从 1 开始,依次类推.

RANK():在结果集中每一条记录所在的排名位置,但排名可能不连续,比如:若同一组内有两个第一名,则下一个名次直接跳至第三名.

DENSE_RANK():功能与 Rank 相似.但排名的数值是连续的,比如:若同一组内有两个第一名,则该组内下一个名次为第二名.

NTILE():依据指定的分组数量将结果集分区,并记录其在组中的位置.(并不是全部的组都有同样的记录数)

3.临时表将结果存储在 tempDB 中.

局部临时表(由#表名定义),仅适用于当前连接和当前登录.关闭连接时删除.

全局临时表(由##表名定义),创建时可用于任何连接在使用它们的最后一个连接关闭时断开.

视图与临时表的区别:空间分配:物理空间的分配不一样,视图不分配空间，临时表会分配空间;虚实:视图是一条预编译的 SQL 语句,并不保存实际数据,而临时表是保存在 tempdb 中的实际的表.即视图是一个快照,是一个虚表,而临时表是客观存在的表类型对象.它们的结构一个是表\一个快照.可以把视图想象成联合表的快捷方式

空间数据库和非关系型数据库:

1.空间索引(B 树索引\哈希索引\R 树索引\位图索引)基本概念:

空间索引是一种用于在空间数据中查找对象的数据结构.常见的空间索引包括：

B 树索引是最常见的索引类型,适用于一维数据的范围查询.它基于 B 树数据结构实现,将数据按照一定的顺序存储在树中,并且支持范围查询和排序.

哈希索引基于哈希表实现,适用于一维数据的等值查询.它可以快速定位数据,但不支持范围查询和排序.

R 树索引是一种空间索引,适用于二维和三维空间数据的范围查询.它基于 R 树数据结构实现,将数据按照空间位置存储在树中,并且支持范围查询和排序.

位图索引是一种基于位运算的索引,适用于高维空间数据的范围查询.它可以将多个属性合并为一个位图,并使用位运算进行查询和操作.

2.空间数据类型

点(Point)、多点(MultiPoint)、线(LineString)、多线(MultiLineString)、面(Polygon)：具有 1 个外环和 0 个或 n 个内环构成)

多面(MultiPolygon)：由多个相离的 Polygon 构成，并没有一个外环。

基本函数

SpatialReference():返回基础坐标系.
Envolope():返回最小边界矩形(MBR).
Export():以不同的表示形式返回几何体
IsEmpty():如果几何体为空集,则返回 true.
IsSimple():如果没有自相交,则返回 true
Boundary():返回几何体的边界.

几何对象存取函数:

获取几何对象的 WKT 描述 ST_AsText(geometry)
获取几何对象的 WKB 描述 ST_AsBinary(geometry)
获取几何对象的空间参考 ID ST_SRID(geometry)
获取点的 X 坐标 ST_X(geometry)
获取点的 Y 坐标 ST_Y(geometry)

空间操作与关系:

ST_DWithin(A,B,D):A 与 B 相距 D
ST_Crosses(A,B):A 线与 B 线相交
ST_Touches(A,B):A 与 B 至少有一个点相交且内部不相交
ST_Contains/**ST_Within**:反函数
ST_Intersects(A,B):图形有共同的交集

几何对象处理函数:

获取两个几何对象间的距离 **ST_Distance**(geometry, geometry)
判断两个几何对象是否相交 **ST_Intersects**(geometry, geometry)
判断两个几何对象的边缘是否接触 ST_Touches(geometry, geometry)
获取几何对象的中心 ST_Centroid(geometry)
面积测量 ST_Area(geometry)
长度测量 ST_Length(geometry)
获取缓冲后的几何对象 ST_Buffer(geometry, double, [integer])

几何操作符:

A 二维相交 B　A&&B
A 包含 B A~B
A 边界在 B 左边 A<<B
A 边界在 B 右边 A>>B
A 边界被 B 包含 A@B

几何量测函数:

ST_Distance_Sphere(point, point):根据经纬度点计算在地球曲面上的距离,单位米,地球半径取值 6370986 米
ST_Distance_Spheroid(point, point): 两个经纬度坐标之间距离,精度较高,效率比 ST_Distance_Sphere 稍低
ST_Distance:根据空间参考计算距离,返回的单位根据投影单位
ST_max_distance(linestring,linestring):量测两条线之间的最大距离
ST_azimuth(geometry, geometry):量测两点构成的方位角,单位弧度

线性参照:

根据 location(0-1)获得该位置的点
ST_line_interpolate_point(linestring, location)
获取一段线
ST_line_substring(linestring, start, end)
根据点到线的最近距离获取 location(0-1)
ST_line_locate_point(LineString, Point)
根据量测值**估获**几何对象 ST_locate_along_measure(geometry, float8)

3.常见空间数据库操作

3. 工作原理：空间数据是使用特定投影的坐标系存储的。该投影使用空间引用标识号（SRID）进行引用 WGS84 坐标系：即地球坐标系，国际上通用的坐标系。（SRID=4326）GCJ02 坐标系：即火星坐标系，WGS84 坐标系经加密后的坐标系。高德地图

BD09 坐标系:即百度坐标系,GCJ02 坐标系经加密后的坐标系。百度独有 Web 墨卡托坐标系（SRID=3857）

4.BASE 理论及分布式系统相关概念:

CAP 理论：一致性(consistency)、可用性(availability)和分区容忍性(partition tolerance)对于一个分布式系统而言，不可能同时满足以上三点；可将 C 理解为副本异质性，所有副本给出结果一致 BASE 理论是指基本可用\软状态\最终一致性.相对于 ACID(原子性\一致性\隔离性\持久性)理论,BASE 理论更适用于分布式系统中的高可用性和可扩展性要求.

基本可用(Basically Available)是指系统必须保证基本的可用性,即在故障或异常情况下,系统仍能够提供基本的服务,而不是完全失效.

软状态(Soft state)是指系统的状态可以根据不同的条件和环境而发生变化,而不是一成不变的硬状态.

最终一致性(Eventually Consistent)是指系统的状态最终会达到一致性,但不一定是实时的.在分布式系统中,由于网络延迟\节点故障等原因,数据可能会存在一段时间的不一致,但最终会达到一致性.

分布式系统是由多个计算机组成的系统,这些计算机通过网络连接在一起,共同完成一个任务.

BASE 思想实际上是 CAP 理论中 AP 的衍伸.它通过牺牲高一致性,保证高可用性和分区容忍性.BASE 思想:基本可用\软状态\最终一致性.BASE 模式指的是一个应用在任意时间首先应该能完成最基本化的工作(即基本可用),并不需要总是一致(即软状态),但最终应该是一致(即最终一致性)的.ACID 和 BASE 应该被看作同一范畴内的互相补充品,而不是替代品

关系模型由关系数据结构、关系操作集合、关系完整性约束三部分组成

5.NoSQL 非关系型数据库. Not Only SQL 的缩写是对不同于传统的关系型数据库的数据库管理系统的统称.NoSQL 用于超大规模数据的存储. NoSQL 可分为四类：键值存储数据库、列存储数据库、文档型数据库和图形数据库

Operator	Description	Operator	Description
=	等于	<=	小于或等于
<> or !=	不等于	BETWEEN *a* AND *b*	位于包含范围内
>	大于	LIKE	匹配一种字符串模式
<	小于	IN (*a, b, c*)	等于多个可能值中的一个
>=	大于或等于	IS NULL *or* IS NOT NULL	与null做比较(缺失数据)

NoSQL 共同特征：不需要预定义模式\无共享架构\弹性可扩展\分区\异步复制\BASE

NoSQL 适用情况：数据模型比较简单\需要灵活性更强的 IT 系统\对数据库性能要求较高\不需要高度的数据一致性

文档数据库基本概念:文档数据库是一种基于文档存储模型的数据库.不需要预定义模式,可以灵活地存储和查询数据.文档数据库将数据存储为文档,每个文档以 JSON 或 XML 格式,具有自我描述性和可扩展性.文档数据库还支持嵌入式文档和数组,可以更好地表示复杂的数据结构.(数据类型丰富,功能丰富,容易扩展,性能卓越,管理简便) **比键值数据库的查询效率更高**

Python 基础

1.**基本数据结构：**int、float、bool、str（字符串不能改变）

列表是一种**有序**的可变序列,可**包含不同类型**的元素.创建列表:方括号[]或 list().支持索引\切片\拼接+\添加\删除和修改元素.索引:

- len(list)/max(list)/min(list)/list(seq)
- list对象方法
  - list.append(obj) #将对象obj加入到列表最后
  - list.count(obj) #计算某一个对象在列表中出现的次数
  - list.extend(seq) #将序列seq追加到列表末尾 #append与extend有什么区别？
  - list.index(obj) #元素obj在列表中的第一个索引值
  - list.insert(index,obj) #在位置index插入元素obj
  - list.pop() #将最后一个元素移除并返回
  - list.remove(obj) #移除obj元素的第一个匹配项
  - list.reverse() #将列表中的元素反向
  - list.sort(key=None,reverse=False) #列表元素排序，元素必须是可比较的
  - list.clear() #清空
  - list.copy() #复制列表

正向\反向. **append 是整体加入，extend 是打散加入**

元组:用小括号括起来的元素,用逗号分割,元组是**不可变/不可修改**的.创建元组:圆括号 () 或 tuple() .支持索引和切片操作,不支持添加\删除和修改元素,只能进行查询操作.

集合:不可重复元素,用{}或 set()创建,空集只能 set()创建.方法:remove**discard 与 remove 功能相同,元素不存在不报错**\差并交补(两集合中不同元素)\s1.issubset(s2)#s1 是否是 s2 的子集

字典是无序的键值对集合,创建字典:{} 或 dict().**键必须是唯一的,但值可以重复**,通过键来访问\添加\删除和修改值,是可变的,查询\修改操作.使用键来访问和修改字典中的值.方法:items\keys\pop\len.

如果不知道某个 key 是否存在于字典中,则需要先进行判断;在修改时,如果键存在,则修改原值为新值;否则在字典中加入新的键值对

2.**基本运算符：**算术运算符(+\-*\/ \//整除,向负无穷取整\%**等)\比较运算符(==\!=\>\<等)\逻辑运算符(and\or\not\is 判断引用是否为同一个对象)和赋值运算符(=, +=, -=) **5//2=2,-5//2=-3** ;==是比较值是否相等，is 是比较标识 id 是否相同（标识相同值一定相等）

['A','B','C','D','E','F','G','H','I','J','K','L','M','N']

A[1:4] ['B','C','D']　A[:4] ['A','B','C','D']　A[-2] ['A','B','C','D','E','F','G','H','I','J','K','L']　A[::-1]逆序输出

list1[:n]表示从列表list1第0个元素开始截取至第n个元素(不包括第n个元素)
list1[n:]表示从列表list1第n个元素开始截取至最后一个元素(包括第n个元素)
list1[a:b:c]表示从列表list1第a个元素开始截取至第b个元素步长为c(不包括第b个元素)
list1[-n]表示截取倒数第n个元素

3.Numpy 和 pandas 库中基本函数的应用

Numpy 是用于处理多维数组的库,提供了用于创建\操作和计算数组的函数,元素类型必须是一致的

np.array():创建一个 Numpy 数组. **np.sort():**排序,默认小到大 **np.concatenate():**连接多个数组,axis=0,逐行拼接

index 和切片\加法\求和\乘法\矩阵乘法 np.dot()

iter():生成可循环序列的循环算子,提供了访问序列的途径

enumerate(s,start=):将一个可遍历的数据对象(如列表\元组或字符串)组合为一个索引序列同时列出数据和数据下标

np.arange():创建一个等差数列.

Pandas 是一个处理二维表格数据的 Python 库,存储不同类型的数据,它提供用于创建\读取\修改和计算 DataFrame 的函数.

DataFrame 创建:字典\二维数组　一维数据:pd.Series(data,index,dtype,name,copy)

4.Pandas.DataFrame 数据表的基本操作,包含缺失值处理\索引\排序等等

缺失值处理: df.dropna()方法来删除包含缺失值的行或列,df.fillna()方法来填充缺失值,df.interpolate()方法来插值填充缺失值.

索引: Pandas .loc 属性来根据标签进行索引,.iloc 属性来根据位置进行索引,用布尔索引来根据条件进行索引.

排序:Pandas df.sort_values()方法来根据指定的列对 DataFrame 进行排序.例如,df.sort_values('A') 将返回一个新的 DataFrame,其中所有行都按照列 A 的值进行升序排序.　**筛选:**使用布尔索引来根据条件筛选

合并:pd.concat()或 pd.merge() 函数来合并两个或多个 DataFrame.pd.concat([df1, df2]) 将按行合并,而 pd.merge(df1, df2, on='key') 根据指定的键合并　**分组:**df.groupby() 方法来根据指定的列分组.

机器学习

1.回归问题与分类问题的区别

回归问题与分类问题的**区别在于输出变量的类型**.定量输出称为回归,或者说连续变量预测;定性输出称为分类,或者说离散变量预测.另一个区别在于**损失函数的形式不同**.回归问题通常使用均方误差作为损失函数,而分类问题通常使用交叉熵损失函数.

分类:两类或多类,返回每个类的置信度(模型输出).模型的预测会产生一个分离类的决策边界.

回归问题是定量问题,分类问题是定性问题.**回归与分类的根本区别在于输出空间是否为一个度量空间.**

2.监督学习、无监督学习与评价指标

监督学习：利用一组已知类别的样本调整分类器的参数,使其达到所要求性能的过程,也称为监督训练或有教师学习.

无监督学习：根据类别未知(没有被标记)的训练样本解决模式识别中的各种问题（聚类、降维）

评价指标：准确率(Accuracy)是分类正确的比例(正类占总);**精准率(Precision)**又叫查准率,它是指被预测为正样本的检测框中预测正确的占比.**召回率(Recall)**又叫查全率,含义是在实际为正的样本中被预测为正样本的概率.**F1 是精确率和召回率的调和均值**,同时考虑了查准率和查全率,让二者同时达到最高,取一个平衡,正确预测为正类占全部实际为正类的比例.

偏差(bias)度量了模型的预测值与真实值之间的偏离程度.偏差越大,表示模型的预测值与真实值之间的偏离程度越大,模型的拟合(泛化)能力越差.

方差(variance)度量了模型对不同训练数据的敏感性.方差越大,表示模型对不同训练数据的预测结果差异越大,模型的泛化能力越差.

离差：实际观察值与其平均值的偏离程度，只与样本有关，与模型无关

残差指观察值与预测值之间的差异 TSS=ESS+RSS(总体平方和=回归平方和+残差平方和) R^2=ESS / TSS ESS 为预测值与观察值平均之差；RSS 为观察值与预测值之差

Error=noise^2+bias^2+variance 所有的训练过程中,都要隐藏测试集;小数据集:多次训练测试分割,交叉验证

3.**范数**

L0 范数:非零元素个数;

L1 范数:各元素绝对值之和(LASSO 回归)L1 范数是指向量中各个元素绝对值之和.它可以使某些参数变为 0,从而实现稀疏表示 1.**L1 范数可以产生稀疏权值矩阵,即产生一个稀疏模型,把不重要的特征直接置零,也就是特征自动选择,所以 L1 是一个天然的特征选择器;**

L2 范数:各元素平方和的 1/2 次方(岭回归)它的作用是减小模型所有参数大小,可以防止模型过拟合,**它将所有参数都缩小,但不会使参数变为 0,因此可以用来进行参数的平滑化**

∞范数:所有元素中绝对值最大的值

Regularization（正则化）

λ= regularization strength (hyperparameter)

$$L = -\frac{1}{N}\sum_{i=1}^{N}(\hat{y}-y)^2 + \boxed{\lambda R(W)}$$

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n}|x_i| \qquad \|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n}x_i^2} = \sqrt{\mathbf{x}^T\mathbf{x}} \qquad \|\mathbf{x}\|_\infty = \max_i|x_i|$$

In common use:

L2 regularization	$R(W) = \|w\|_2$ （ ridge regression: 岭回归）
L1 regularization	$R(W) = \|w\|_1$（Lasso regression）
Elastic net (L1 + L2)	$R(W) = \alpha\|w\|_2 + \|w\|_1$

4.**梯度下降算法的基本概念和运算逻辑**

梯度下降算法是常用的优化算法,用于最小化目标函数.基本思想是通过计算目标函数的梯度(即偏导数),沿着梯度负方向更新参数,以最小化目标函数.梯度下降算法的运算逻辑如下:

(1)首先定义一个**目标函数(也称为损失函数或代价函数)**,用于衡量模型的预测值与真实值之间的差距.

(2)然后计算目标函数关于模型参数的梯度(即偏导数).梯度表示了目标函数在当前参数取值下的变化方向.

(3)接下来沿着梯度的负方向更新参数.这意味着我们将参数沿着使目标函数下降最快的方向进行更新.

(4)重复上述步骤,直到目标函数收敛到一个局部最小值.

梯度下降算法**不能保证找到全局最小值,只能找到局部最小值**.因此,初始化参数的选择和学习率的设置都会影响算法的收敛结果.（与正规方程比较，正规方程特征量 n 很大时计算代价大，且只适用线性模型，不适合逻辑回归模型等）

5.**线性回归：**

基本原理是通过**最小化残差平方和**来拟合数据,从而得到一个线性模型.

(1)定义线性模型,用于描述自变量和因变量间关系.如 y = wx + b,其中 w 和 b 是待求的参数.(2)定义损失函数,衡量模型预测值与真实值之间的差距.通常使用**均方误差**作为损失函数.(3)使用优化算法(如梯度下降)来最小化损失函数,从而求解模型参数.(4)使用求解出的模型参数来预测新数据点的因变量值.

线性回归的结果通过**回归系数**来解释.回归系数表示了自变量与因变量之间的关系.例如,在一元线性回归中,回归系数 w 表示了自变量 x 每增加一个单位时,因变量 y 的变化量.如果 w 为正,则说明 x 与 y 呈正相关;如果 w 为负,则说明 x 与 y 呈负相关.

6.**Logistics 逻辑回归**

Logistic 回归是一种分类算法,**解决二分类问题**.基本原理是:通过将线性回归模型的输出值通过一个 Logistic 函数进行映射,将连续值转换为概率值,然后根据概率值进行分类.

(1)定义线性模型(2)然后将线性模型的输出值通过 Logistic 函数(sigmoid(二分类)/softmax（多元分类）) 进行映射,将连续值转换为概率值.

(3)定义损失函数,用于衡量模型预测值与真实值之间的差距.Logistic 回归通常使用**交叉熵损失函数**作为损失函数.

(4)使用优化算法(例如梯度下降算法)来最小化损失函数,从而求解模型参数.(5)最后使用求解出的模型参数来预测新数据点的类别.

Logistic 回归模型可以输出预测概率,表示样本属于正例的概率.优化目标函数为似然函数（对概率做处理且取对数）

7.**KNN：**

K 最近邻算法((K-Nearest Neighbors)),是一种基于实例的学习，或称为惰性学习，其中函数只在局部近似，所有计算都推迟到分类进行时。它的基本原理是:对于一个新的数据点,通过计算它与训练集中所有数据点的距离,然后找到距离最近的 K 个数据点.这 K 个数据点中出现最多的类别就是新数据点的预测类别.输出的是类的隶属度。输出是对象的属性值。该值是其 k 个最近邻值的平均值。

所有实例都对应于 n 维欧氏空间中的点；**高计算负担**；通过比较不同点的特征向量进行分类；目标函数可以是离散函数或实值函数；高噪声训练数据和复杂目标函数的高效归纳推理方法。

需要注意的是,**KNN 算法对于 K 值的选择非常敏感.如果 K 值过小,则容易受到噪声数据的影响;如果 K 值过大,则容易忽略真实类别之间的差异.**因此,选择合适的 K 值是应用 KNN 算法时需要考虑的重要因素.

8.**Support Vector Machine (SVM):** 支持向量机，是监督学习模型，包括了对应的算法，可以用于分类和回归分析。支持向量机模型是将实例表示为空间中的点，通过映射使单独类别的实例被一个尽可能宽的间隔完全分割。然后，新的例子被映射到同一个空间，并根据它们落在差距的哪一边被预测为属于一个类别。SVM 适合**中小型数据样本、非线性、高维**的分类问题

①损失函数：hinge loss

②处理非线性可分的情况是选择了**核函数（kernel）**，降低计算的复杂度，通过将数据映射到高位空间，来解决在原始空间中线性不可分的问题。（而核函数的选择包含两部分工作：一是核函数类型的选择，二是确定核函数类型后相关参数的选择。）

③与 LR 对比：logic 能做的 svm 能做，但可能在准确率上有问题，svm 能做的 logic 有的做不了。本质是损失函数不同，两个模型对数据和参数的敏感程度不同（SVM 只考虑分类边界线附近，支持向量；LR 受所有数据点影响），SVM 基于距离分类 LR 基于概率分类

9.**神经网络**

BNN vs ANN 体细胞-节点\树突-输入\突触-权重或连接\轴突-输出

神经网络模型：输入层（+1 圆为偏置单位对应于截距项）、隐藏层、输出层

激活函数σ模拟了神经元在到达一定阈值后兴奋的特点一般为非线性，因为这样才可能表现出复杂的非线性关系，否则神经网络只有较弱的线性拟合能力

$$z^{(l+1)} = W^{(l)}a^{(l)} + b^{(l)}$$
$$a^{(l+1)} = f(z^{(l+1)})$$

$$W_{ij}^{(l)} = W_{ij}^{(l)} - \alpha\frac{\partial}{\partial W_{ij}^{(l)}}J(W,b)$$
$$b_i^{(l)} = b_i^{(l)} - \alpha\frac{\partial}{\partial b_i^{(l)}}J(W,b)$$

前向传播

反向传播调整神经网络的权值，最小化网络的总均方误差（MSE）BP 算法，也叫误差反向传播算法

CNN 卷积神经网络一种前馈神经网络，通常由卷积层，池化层和全连接层（对应经典的 NN）组成。卷积层负责提取图像中的局部特征；池化层用来大幅降低参数量级(降维)；全连接层类似传统神经网络的部分，用来输出想要的结果。**CNN 参数共享的好处：**大幅度减少了参数量，极大的提升了运算效率；解决了很多图像识别中目标的位置不固定等问题。图像处理

RNN 循环神经网络针对 CNN 中无法对时间序列上的变化进行建模的局限，为了**适应对时序数据**的处理。文本处理、语音处理、视频分类、图像标注

LSTM 为了解决 RNN 中时间上的梯度消失，通过门的开关（遗忘门、输入门和输出门）来控制长期和短期记忆，并防止梯度消失和爆炸。翻译语言、控制机器人、图像分析、文档摘要、语音识别图像识别、手写识别、控制聊天机器人、预测疾病、点击率和股票、合成音乐

10.常见损失函数\激活函数的概念及形式.

损失函数(Loss Function)是用于衡量模型预测值与真实值之间的差距.常见的损失函数有:

均方误差(MSE):用于回归问题,预测值与真实值之差的平方和的均值.

交叉熵损失(Cross-Entropy Loss):用于分类问题,定义为真实类别的对数似然的相反数.

This is called ***Cross-Entropy Loss*** (n is the number of observations):

$$L = -\sum_{i}^{n} y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)$$

Hinge 损失(Hinge Loss):定义为 max(0, 1 - y * f(x)),其中 y 为真实类别标签,f(x)为模型预测值.

$$y = \begin{cases} -1 & \text{if } x \notin C \quad f(x) \geq 1 \text{ for } x \in C \\ 1 & \text{if } x \in C \quad f(x) \leq -1 \text{ for } x \notin C \end{cases} \qquad yf(x) \geq 1$$

激活函数(Activation Function)用于增强神经网络非线性拟合能力的函数.常见的有:

Sigmoid 函数:定义为 f(x) = 1 / (1 + exp(-x)),取值范围为(0, 1).

Tanh 函数:定义为 f(x) = (exp(x) - exp(-x)) / (exp(x) + exp(-x)),取值范围为(-1, 1).

ReLU 函数:定义为 f(x) = max(0, x),取值范围为[0, +∞).

11.神经网络参数训练过程可能出现的问题

梯度消失:指在反向传播过程中,梯度逐渐变小,最终接近于 0.这会导致模型参数更新缓慢,训练困难.梯度消失问题通常出现在深层神经网络中,尤其是用 Sigmoid 或 Tanh 作为激活函数时.解决方法:**使用 ReLU 等不饱和激活函数**\使用残差连接\使用预训练\使用长短期记忆或门控循环单元架构 LSTM（通过门控机制对抗梯度消失）\正确初始化权重矩阵 W 等.

梯度爆炸:指在反向传播过程中,梯度逐渐变大,最终变得非常大.这会导致模型参数更新过快,甚至发散.梯度爆炸问题通常出现在循环神经网络中.解决方法有:使用梯度裁剪\使用 LSTM 或 GRU 等门控循环单元等.

过拟合:指模型在训练集上表现得很好,但在测试集上表现不佳.这意味着模型过于复杂,以至于记住了训练数据中的噪声．低偏差和高方差；低训练误差和高测试误差；解决方法有:将数据拆分成多个训练-验证-测试集**正则化（调整验证集上的超参数以简化模型）**\使用 Dropout\增加数据量等.

欠拟合:指模型在训练集和测试集上都表现不佳.这意味着模型过于简单,无法捕捉数据中的关键信息．高偏差和低方差；高训练误差和高测试误差.解决方法有:增加模型复杂度\减少正则化系数等.

1.**线性回归分析的目的**是估计自变量对因变量的影 2.**残差**指观测值与预测值之间的差异

3.**R 方（决定系数）**指回归平方和占总平方和的比例

4.过拟合的解决方法有正则化，减少特征维度，清洗数据集，欠拟合的解决方法有添加多项式特征

5.因变量离散可用逻辑回归 6.逻辑回归的响应变量是一个对数优势比

7.LASSO 可用于变量选择 8.AIC 值越小越好

9.神经网络激活函数的作用：用于增加神经网络的非线性

10.深度学习中常用的优化算法：随机梯度下降法

11.**卷积层**的作用：提取图像特征 12.相较于简单的循环神经网络，长短时记忆 LSTM 单元的作用：解决梯度消失问题

13.**池化层**的作用是压缩图像尺寸，避免过拟合 14.交叉熵损失函数常用于分类任务

Batch:~Size 是一次训练所选取的样本数，影响模型的优化程度和速度 GPU 内存的使用情况。没有 BatchSize，适用小样本数据库;Batch size=1，梯度变来变去，网络很难收敛;Batch Size 增大，梯度变准确;Batch Size 增大了要到达相同的准确度，必须要增大 epoch.

Epoch:(轮/轮次将所有训练样本训练一次的过程所有训练样本在神经网络中都进行了一次正向传播和一次反向传播 Iteration:(一次迭代)训练一个 Batch 就是一次 Iteration